Versión 2.4 — 3 de octubre de 2026

NIVEL 2 COMPLETO (0.1 a 2.6) — Nivel 3 abierto: las cifras (3.1)*

> **Arreglo de esta versión (fallo que encontraste tú, y era el más serio hasta ahora):** en la unidad
> 2.4, `dame: tres` **se inventaba los objetos** (los fabricaba el propio sistema). Tú preguntaste
> **de dónde los sacaba**, y tenías razón: **no hay de dónde**, porque el sistema no tiene un mundo.
> Eso rompía la regla del proyecto de que toda respuesta se rastre a un **dato escrito**. Arreglado
> **con tu idea**: `dame` recibe el montón igual que `tiene` → `dame: a,b,c,d,e / tres`. El sistema
> ya no crea objetos: **elige de los que le das**. Y si no le das montón, lo dice.

Novedad de la versión 2.4: la unidad 5.2 — LA SUMA (los signos)

Ya están los signos. Pero con la idea clara que dice la didáctica: la suma existe antes que su símbolo. Primero se aprende juntando; el signo llega después a escribir eso.

```
sumar: a,b,c / d,e     ->  3 + 2 = 5
```

El = NO es «pon el resultado»

Es la trampa principal, y el sistema lo dice en cada respuesta:

```
QUE SIGNIFICAN LOS SIGNOS:
  +   se lee 'mas' y significa JUNTAR lo de los dos lados
  =   se lee 'igual a' y significa QUE LAS DOS COSAS SON LO MISMO
      (OJO: el = NO es 'haz la cuenta y pon el resultado'.  Es que 3 + 2 y 5 son la
       misma cosa dicha de dos maneras.  Por eso 5 = 3 + 2 es igual de cierto.)
```

Y la suma tiene DOS estructuras (esto no me lo esperaba)

Hay dos formas de contar la misma suma, y las dos importan:

```
sumar:  a,b,c / d,e    ->  AGREGACION   "en un lado hay tres y en el otro dos; juntos son cinco"
anadir: a,b,c / d,e    ->  AUMENTACION  "PRIMERO habia tres; LUEGO se anaden dos; AHORA hay cinco"
```

El número final es el mismo — hay una prueba que lo comprueba — pero **la historia es distinta**: en la agregación los dos montones **ya estaban**; en la aumentación el montón **crece**.

Los términos

```
los SUMANDOS son tres y dos   (los numeros que se juntan)
la SUMA es cinco              (el resultado)
```

Y la suma se comprueba partiendo el resultado

```
VERIFICACION (por la vuelta, unidad 5.1):
  partiendo 'cinco' sale: cero y cinco, uno y cuatro, dos y tres
  la pareja de los sumandos ('tres y dos') esta entre ellas: SI
```

Si 3 + 2 = 5, al partir 5 **tiene que aparecer** «tres y dos». Y aparece. Ahí está la suma y la resta

empezando a ser la una la vuelta de la otra.

Condiciones de aplicación (declaradas, no escondidas)

```
sumar: a,b,c,d,e,f / g,h,i,j,k   -> no_alcanza
  CONDICION DE APLICACION: no se suma mas alla de lo que se cuenta.
```

Y en el límite exacto **sí suma**: `sumar: a,b,c,d,e,f / g,h,i,j` → `6 + 4 = diez` (con palabra, porque la cifra del diez no existe).

Respuestas nuevas para tu lista

Respuesta	Cuándo
`3 + 2 = 5`	(5.2) la expresión de la suma
`3 + 7 = diez`	(5.2) cuando el total es diez: se da la palabra
`no_alcanza`	(5.2) la suma pasa de diez

(versión 2.3) la unidad 5.1 — COMPONER Y DESCOMPONER

Se abre el Nivel 5: las OPERACIONES. Y se abre por donde tú decidiste — no por la suma, sino por lo que la sostiene: **las partes y el todo**.

```
descomponer: a,b,c,d,e        -> 0+5,1+4,2+3        (cinco, de TRES maneras)
componer: a,b / c,d,e         -> cinco              (dos y tres son cinco)
```

Se ve el todo y sus partes a la vez

```
EL TODO (la coleccion entera):
  cantidad:  *****     palabra: cinco     cifra: 5

LAS PARTES, todas las maneras (3):
  1. cero   y cinco    (0 y 5)     (vacio) | *****
  2. uno    y cuatro   (1 y 4)     * | ****
  3. dos    y tres     (2 y 3)     ** | ***
```

TRES cosas que conviene que sepas

1. Cinco se parte de TRES maneras, no de seis. Porque 2+3 y 3+2 son **la misma** partición — el orden no cambia la cantidad (eso ya lo sabía desde la unidad 2.5). El sistema **no la repite** y te lo explica en cada respuesta.

2. El cero es una parte válida (`0+5`)… pero **el cero no se puede partir**:

```
descomponer:        -> el_cero_no_se_parte
  De cero no hay nada que partir.  El cero SI puede ser una PARTE, pero no un TODO.
```

3. Aquí NO se escribe el signo +. Se dice con palabras: *«tres y dos son cinco»*. El signo es la unidad siguiente.

Y la vuelta lo verifica

Juntar lo que has partido tiene que devolver el todo. El sistema lo comprueba y lo dice:

```
componer: a,b / c,d,e
RESULTADO: cinco
  'dos' y 'tres' son 'cinco'   (con las cifras: 2 y 3 son 5)
  VERIFICACION (por la vuelta): partiendo 'cinco' en esas dos partes, vuelven a salir
  las mismas: SI.
```

Eso es justo lo que va a sostener que la **suma y la resta sean la una la vuelta de la otra**.

Respuestas nuevas para tu lista

Respuesta	Cuándo
`N maneras`	(5.1) al descomponer: cuántas maneras hay de partir en dos
`el_cero_no_se_parte`	(5.1) `descomponer:` con la colección vacía
el total en palabras	(5.1) al componer, p. ej. `cinco`
`no_alcanza`	(5.1) más de diez objetos, o al juntar pasar de diez

(versión 2.2) la unidad 4.2 — COMPARAR NÚMEROS

Ya está el Nivel 4 entero: comparar **cantidades** con palabras (4.1) y comparar **números** con **símbolos** (4.2). Y las dos mitades dicen lo mismo.

```
comparar_cifras: 3 / 2   ->  >
comparar_cifras: 2 / 3   ->  <
comparar_cifras: 4 / 4   ->  =
```

De dónde sale que un número sea mayor

No del dibujo de la cifra. Del **orden**: *en la secuencia de números, el que viene **después** es el **mayor***. Y el sistema te lo enseña cada vez:

```
DE DONDE SALE (el orden de la secuencia de numeros, unidad 3.3):
  cero, uno, dos, tres, cuatro, cinco, seis, siete, ocho, nueve
  el 'tres' esta en el sitio 3 y el 'dos' en el sitio 2: el que viene DESPUES es el MAYOR.
Y CUADRA CON LAS CANTIDADES (unidad 4.1): 3 cosas frente a 2 cosas.
  los dos caminos dicen lo mismo: SI
RESULTADO: >
  3 > 2   (tres es mayor que dos)
  La boca del simbolo (>) mira al numero MAYOR, que es el 3.
```

Fíjate en la línea clave: `3 > 2` **porque tres cosas son más que dos**. El símbolo no es una regla suelta — es lo mismo que ya sabía decir con palabras.

Vocabulario (y no es un capricho)

De **grupos de cosas** se dice que hay **más** o **menos**. De **números** se dice que uno es **mayor** o **menor** que otro. No es lo mismo *«aquí hay menos»* que *«cinco es menor que tres»*, y el sistema no

las mezcla.

Y ojo al símbolo que más se confunde

> y < son **iguales vistos al revés**. El sistema lo ancla en algo que sí se ve: la boca mira al número mayor. Pruébalo con `comparar_cifras: 2 / 3` y verás la frase.

Respuestas nuevas para tu lista

Respuesta	Cuándo
>	(4.2) la primera cifra es **mayor** (viene después en la secuencia)
<	(4.2) la primera es **menor**
=	(4.2) las dos cifras son la misma
`cifra_desconocida`	(4.2) alguna no es una cifra del 0 al 9 (p. ej. `10`, que son dos símbolos)

(versión 2.1) la unidad 4.1 — COMPARAR CANTIDADES

Se abre un nivel nuevo: las RELACIONES. Y se abre por donde tú decidiste: primero cantidades, después números.

```
comparar: a,b,c / d,e      ->  mas          (la PRIMERA tiene más: tres contra dos)
comparar: a,b / c,d,e      ->  menos
comparar: a,b / c,d        ->  tantos_como  (las dos tienen lo mismo)
```

Se dice con PALABRAS, no con símbolos

Y eso no es una comodidad: es lo que dice la didáctica. Los símbolos =, <, > solo se usan entre cifras (National Academy). Así que aquí se dice `mas`, `menos`, `tantos_como` — los símbolos llegan en la unidad siguiente (4.2), con los números.

Y lo bonito: no hace falta contar para comparar

Para saber cuál tiene más basta **emparejar**: un objeto de una colección con uno de la otra, y mirar **cuál sobra**. Eso el sistema ya lo sabía hacer (es la unidad 2.1). Y encima **verifica contando**:

```
CAMINO 1, EMPAREJAR (uno a uno, SIN contar):
    a            -- d
    b            -- e
    c            -- (no tiene pareja)
CAMINO 2, CONTAR (la verificacion):
    primera: 3 -> tres
    segunda: 2 -> dos
    los dos caminos dicen lo mismo: SI
RESULTADO: mas
  La PRIMERA tiene mas: tres contra dos.
```

Los dos caminos tienen que decir lo mismo. Si no cuadraran, el sistema no elegiría uno: lo diría.

La trampa de la unidad: el TAMAÑO de las cosas no cuenta

Esto es justo lo que más cuesta aprender a un niño (está en la investigación: hay que aprender a **ignorar el tamaño**). Pruébalo:

```
comparar: g1(tamano=grande), g2(tamano=grande) / p1(tamano=pequeno), p2(tamano=pequeno), p3(tamano=pequeno)

  OJO AL TAMANO: hay objetos grande y pequeno.
  El tamano NO cuenta: dos cosas grandes NO son mas que tres pequenas.
  La cantidad son las COSAS, no lo que ocupan.
RESULTADO: menos
```

Dos **grandes** tienen **menos** que tres **pequeños**. (Los atributos son los de la unidad 0.2, así que `tamano=grande` funciona como cualquier otro.)

Y también se compara el cero

```
comparar: /        -> tantos_como  (dos colecciones vacías: cero y cero)
comparar: / a      -> menos        (vacía contra una cosa)
```

Respuestas nuevas para tu lista

Respuesta	Cuándo
`mas`	(4.1) la **primera** colección tiene más
`menos`	(4.1) la primera tiene menos
`tantos_como`	(4.1) las dos tienen lo mismo
`no_alcanza`	(4.1) alguna pasa de diez objetos: no le llegan las palabras
`no_entendido`	(4.1) `comparar: a,b,c` sin la barra: comparar son **dos**

(versión 2.0) la unidad 3.4 — EL PUENTE

Lo mismo se dice de tres maneras, y el sistema sabe ir de cualquiera a cualquiera:

```
cantidad:  * * *      (los objetos)
palabra:   tres       (como se dice al contar)
cifra:     3          (como se escribe)
```

Escribe `caminos` y te enseña **los seis caminos** entre las tres, y en qué orden se aprenden.

El camino que faltaba: de la CIFRA a los objetos

```
dame: a,b,c,d,e / 3      -> coge tres del montón (y te dice que se lo pediste en cifra)
```

Antes había que pedirlo con la palabra (`/ tres`). Ahora vale con la cifra. Ya están **los seis**.

Y lo más bonito: el «de un vistazo» ya no está suelto

Desde el primer día había una unidad (la **1.0**) que sabía reconocer un grupo pequeño **sin contar**,

y llevaba cerrada pero **aislada**: no servía para nada. **Su sitio es este.**

```
EL CAMINO LARGO  -> CONTAR: una palabra por objeto, en orden.  Vale hasta diez.
EL CAMINO CORTO  -> DE UN VISTAZO: reconocer el grupo sin contar.  Solo grupos pequeños.
```

Prueba `subitizar: x.x/.../x.x` y verás que **ahora sí dice la cifra**:

```
es el patron 'esquinas_cuatro'  ->  CUATRO
CAMINO: DE UN VISTAZO (el corto).  NO he contado: he reconocido el dibujo.
RESULTADO: CUATRO
  su CIFRA es: 4
```

Y la regla que los gobierna: **el sistema DICE por cuál ha ido**, y el camino corto **no sustituye** al largo. Cuando no reconoce un dibujo, lo dice — y **no cuenta a escondidas**.

Dos sitios donde el sistema dice «esto no lo tengo»

Porque no todo se puede decir de las tres maneras:

```
puente: a,b,c,d,e,f,g,h,i,j     -> diez
  PUENTE (las representaciones que SI tengo):
    cantidad:  **********
    palabra:   diez
    cifra:     (no la tengo)
```

Se puede **contar** hasta diez, pero **las cifras solo llegan a nueve**: el diez son **dos símbolos** (1 y 0), y eso es el Sistema Decimal, que es **otra cosa**. El sistema da las dos que tiene y dice cuál le falta, en vez de inventarse la tercera.

```
dame: a,b,c,d / 0       -> cero_no_se_coge
  Me pides CERO objetos.  El cero ES una cantidad (unidad 3.2), pero no una cantidad
  que se pueda COGER: dar es coger, y el vacío no se coge.
```

Nuevas respuestas, para tu lista de casos

Respuesta	Cuándo
`puente_sin_cifra` (da la palabra)	(3.4) la cantidad no tiene cifra: el caso del **diez**
`cero_no_se_coge`	(3.4) `dame: ... / 0`: dar es coger, y el vacío no se coge
`los_seis_caminos`	(3.4) al escribir `caminos`

(versión 1.9) la unidad 3.3 — las DOS listas

Aquí se arregló un fallo que cazaste tú. Probaste `anterior: uno` y el sistema dijo «antes no hay nada»… cuando el mismo sistema, en `cuantos:` con la colección vacía, decía `cero`. O las dos cosas son verdad o ninguna.

La causa: al añadir el cero (3.2) quedó en la **tabla de cifras** pero no en la **secuencia**. Dos datos que hablaban del mismo mundo y no coincidían.

El arreglo: hay DOS listas, y son cosas distintas

```
LA LISTA DE CONTAR (para contar cosas; empieza en el primer objeto):
    uno, dos, tres, cuatro, cinco, seis, siete, ocho, nueve, diez

LA SECUENCIA DE NUMEROS (el orden de los números; lleva el cero delante):
    cero, uno, dos, tres, cuatro, cinco, seis, siete, ocho, nueve
```

Escribe `listas` a secas y te enseña las dos con su diferencia.

Y no es una convención nuestra: **la recta numérica empieza en 0; la secuencia para contar empieza en 1.**

Qué orden usa cada lista

- `siguiente:` **y** `anterior:` → la **secuencia de números**. Así que `anterior: uno` → `cero`.
- `recitar:` → la **lista de contar** (es lo que se recita al contar).

Y el sistema DICE en cuál habla, porque dos listas sin decir cuál es peor que una sola:

```
anterior: uno
RESULTADO: cero
  EN LA SECUENCIA DE NUMEROS: cero, uno, dos, tres, cuatro, cinco, seis, siete, ocho, nueve
  (OJO: esto es la secuencia de NUMEROS. En la lista de CONTAR, el uno es el primero:
   contar arranca en el primer objeto, no en el cero.)
```

Resultado nuevo: `sin_numero`

`diez` está en la lista de **contar** pero **no tiene sitio en la de números** (su cifra son dos símbolos: eso es el Sistema Decimal). El sistema lo distingue de «no lo conozco»:

```
siguiente: nueve   -> no_lo_se       (nueve es el último de los números)
siguiente: diez    -> sin_numero     (existe, pero no en los números)
siguiente: once     -> palabra_desconocida  (no lo conozco)
```

Cuidado con los casos que ya tenías

Dos casos viejos quedaron **obsoletos** y se actualizaron: `siguiente: diez` (era `no_lo_se`) y `anterior: uno` (era `no_lo_se`). Si tú tenías alguno apuntado, cambia igual.

(versión 1.8) la unidad 3.2 — el cero

Cuando una colección está vacía, el sistema ya no dice «no se puede decidir»: dice CERO.

```
cuantos:

COLECCION:
    (vacia)

RESULTADO: cero
  la cantidad de un conjunto vacio es CERO, y su cifra es 0.
  (OJO: esto NO es 'no se sabe'. Es una cantidad: ninguno.)
```

Y lo importante: **no es que antes se equivocara, es que le faltaba el cero.** Aprenderlo ha **ampliado** lo

que sabe, sin romper nada.

La trampa: cero NO es «no se sabe»

- **cero** → sí se sabe: no hay ninguno. Es **una cantidad**.
- **no se sabe** → falta un dato, o no hay contexto.
- **no le alcanza** → se sabe que hay alguno, pero no le llegan los símbolos.

Decir «cero» cuando no se sabe sería inventarse una respuesta.

Y sabe que hay dos ceros

El **cardinal** (ninguno), que es este; y el **posicional** (el 0 que marca un sitio, como en 10), que es del Sistema Decimal y **no** está aquí: `palabra: 10` → `cifra_desconocida`.

Un límite declarado, con prueba

El cero entra **solo donde la pregunta es de cantidad** (`cuantos:`, `cifra_de:`). En `tiene: ¿tiene N?` y `misma_cantidad: ¿son iguales?`, el vacío **sigue** diciendo `no se puede decidir`. Y hay dos pruebas que lo fijan, para que no se cambie por descuido.

(versión 1.7) la unidad 3.1 — los símbolos (LAS CIFRAS)

Hasta ahora las cifras estaban prohibidas a propósito: todo se había hecho con palabras («tres»). Esta unidad introduce el **3**.

Y son **tres representaciones de lo mismo**:

```
cifra_de: a,b,c

  PUENTE (tres representaciones de lo mismo):
    cantidad:  ***
    palabra:   tres
    cifra:     3

RESULTADO: 3
```

- `cifra: tres` → 3 (de la palabra a la cifra)
- `palabra: 3` → `tres` (de la cifra a la palabra)
- `cifra_de: a,b,c` → 3 (de la **cantidad** a la cifra: el camino completo)

Y sabe dónde acaba (que es tan importante como lo que sabe)

```
cifra: diez    -> sin_cifra          («diez» es palabra conocida, pero su cifra son DOS símbolos)
palabra: 0     -> cifra_desconocida  (el cero es la unidad siguiente)
palabra: 10    -> cifra_desconocida  (dos símbolos)
cifra_de: (diez objetos) -> no_alcanza_las_cifras
```

Ninguna de esas es un fallo: es el sistema diciendo **hasta dónde llega** en vez de inventarse una cifra.

El orden de aprendizaje lo dice la investigación

Benoit et al. (2013): a los 3 años se pasa bien de **colección a palabra** pero mal a cifras; a los 4 ya se pasa a cifras; a los 5, entre las tres. El orden es **palabra→colección**, luego **cifra→colección**, y **por último palabra→cifra** — porque *las palabras se las lleva el viento en cuanto se dicen*, mientras que las colecciones y las cifras se quedan a la vista.

(versión 1.6) la unidad 2.6 — abstracción

Se puede contar CUALQUIER COSA, y la prueba es esta: dos colecciones de cosas completamente distintas pueden tener la misma cantidad.

```
misma_cantidad: silla_1,silla_2,silla_3 / sonido_1,sonido_2,sonido_3

  CUENTO LA 1: la ultima palabra es tres
  CUENTO LA 2: la ultima palabra es tres

  RESULTADO: SI, las dos tienen la misma cantidad
```

Fíjate en lo que eso significa: **se puede comparar una silla con un sonido**. Eso solo tiene sentido si la cantidad se ha desprendido de qué son las cosas.

Y lo dice la investigación: niños de **2-3 años** contaron objetos, acciones y **sonidos**; contaron mejor los objetos, pero **la mayoría también contaba acciones y sonidos**.

Un detalle que enlaza con la 0.2: una colección **sin ningún atributo común** (`silla, perro, idea`) también tiene cantidad. **Contar no necesita un grupo: necesita entidades distinguibles.**

(versión 1.5) la unidad 2.5 — el orden no importa

Se puede empezar a contar por donde sea y en la dirección que sea: la cantidad sale la misma. La cantidad es una propiedad **del conjunto**, no del recorrido.

```
orden: a,b,c

  ORDEN 1 (como me lo das):
    a -> uno ; b -> dos ; c -> tres      -> la ultima palabra es tres
  ORDEN 2 (al reves):
    c -> uno ; b -> dos ; a -> tres      -> la ultima palabra es tres

  COINCIDEN: SI   (el orden de los OBJETOS no importa)
```

Y no se limita a decir «sí»: **hace los dos conteos y te los enseña**.

La distinción que trae esta unidad (y es la más útil hasta ahora)

Hay **dos clases de reglas** al contar:

- **LÓGICAS** (obligatorias): una palabra por objeto; las palabras siempre en el mismo orden; la última palabra es la cantidad. **Si se rompen, el resultado no vale.**
- **CONVENCIONALES** (opcionales): por dónde empiezas, en qué dirección, si vas en fila. Romperlas no cambia nada.

Lo dice la investigación (Gelman & Gallistel 1978; Lago, Escudero & Dopico 2021).

Y por eso sabe juzgar un conteo ajeno: pseudoerror contra compensación

```
conteo: c,a,b / uno,dos,tres             -> VALIDO
             (empezó por el tercero: es una regla CONVENCIONAL, y está permitido)

conteo: a,b,b,c / uno,dos,tres,cuatro    -> objeto_repetido
             (contó b dos veces: eso rompe el 1-a-1, que es LÓGICA. Aunque el total cuadre, NO vale)
```

Lo de `a,b,b,c` es el caso más bonito: el conteo **acaba en «cuatro»**, que es el número correcto de objetos… y aun así **no vale**, porque contó uno dos veces y otro ninguna. Esa es la diferencia entre «acertar el número» y «contar bien».

Novedad de la versión 1.3: la unidad 2.4 — cardinalidad

El sistema puede producir un conjunto con la cantidad que le pidas. Y eso es el **principio**, no la regla de la 2.3.

```
dame: a,b,c,d,e / tres

MONTON (lo que me dan, no me lo invento):
    a
    b
    c
    d
    e
CANTIDAD QUE ME PIDEN: tres
  COJO (3 objetos):
    a          -> uno
    b          -> dos
    c          -> tres
  DEJO (no me los llevo): d, e

  VERIFICACION (cuento lo que he cogido): la ultima palabra es 'tres'
  cuadra con lo pedido: SI
```

Por qué dar es más difícil que reconocer

Reconocer una cantidad (conjunto → palabra, la 2.3) es más fácil que **producirla** (palabra → conjunto). En los estudios hay niños que cuentan bien hasta cinco y **no saben dar cinco objetos**. Por eso esta tarea —**Give-N**— es la que distingue de verdad, y es la que usa esta unidad.

El montón lo das tú (el sistema no tiene un mundo)

El sistema **no tiene objetos propios**: no hay de dónde sacarlos. Así que el montón se lo das tú, con la **misma forma** que en `tiene:` — el montón antes de la barra, la cantidad después:

```
dame:    a,b,c,d,e / tres     ->  de estos cinco, dame tres
tiene:   a,b,c / tres         ->  estos tres, tienen tres?
```

Y el sistema **elige** del montón: no inventa nada.

Cuatro cosas que hace, y todas salen de la investigación

- **Se verifica a sí mismo.** Cuenta lo que ha dado y comprueba que cuadra. (En los estudios, pedir que se rechequee mejora los resultados.)
- **Es un *counter*, no un *grabber*.** No agarra un puñado y dice una cantidad: cuenta.
- **No se lo inventa.** `dame: once` → `palabra_desconocida`.
- **Y "¿tiene N?" confirma o rechaza**: `tiene: a,b,c / tres` → **sí**; `tiene: a,b,c / cinco` → **no**, y dice que tiene tres.

La prueba que más vale (coherencia para TODA la lista)

Para cada palabra: `dame: N` → contar lo dado → tiene que dar **N**. De «uno» a «diez», cuadra.

```
OK  dame:uno   ->  1 objeto  -> cuantos: uno
OK  dame:dos   ->  2 objetos -> cuantos: dos
...
OK  dame:diez  -> 10 objetos -> cuantos: diez
```

Las dos direcciones están de acuerdo para **toda** la lista. Eso es el principio funcionando.

Novedad de la versión 1.2: la unidad 2.3 — cuántos hay

El sistema ya dice cuántos hay.

```
cuantos: silla_1, silla_2, silla_3

COLECCION:
    silla_1
    silla_2
    silla_3
  RECUENTO (una palabra por objeto, en orden):
    silla_1    -> uno
    silla_2    -> dos
    silla_3    -> tres

  la ULTIMA palabra del recuento es: tres
RESULTADO: hay_cantidad
```

La respuesta es **la última palabra** — y el sistema lo dice así, no dice «sé que hay tres».

Por qué esa frase tan retorcida (y no es un capricho)

La investigación separa dos cosas que parecen la misma:

- **La regla de la última palabra**: «la respuesta a ¿cuántos hay? es la última palabra». Es un **procedimiento**. Se puede aplicar bien sin entender nada.
- **El principio de cardinalidad**: «esa palabra **es** la cantidad del conjunto». Es el **principio**.

El dato que obliga a separarlas: en un estudio con **188 niños de 4 años y medio**, 42 (un 22,3%) acertaban **diciendo la última palabra** y fallaban la correspondencia uno a uno al contar. O sea, se puede acertar aquí sin entender lo de abajo.

Así que la 2.3 es **la regla**, y la **2.4 será el principio**. No se cuela.

Y las cosas que NO hace

- **No cuenta dos veces.** Los niños, al oír «¿cuántos hay?», muchas veces **vuelven a contar**. El sistema no: ya contó, y da la respuesta.
- **No se inventa la cantidad** si hay más objetos que palabras conoce: `no_alcanza_la_secuencia`.
- **No cuenta dos veces el mismo objeto**: `cuantos: a,a,a,a,a` → **uno**.
- Y tu caso: `cuantos: casa(color=blanco), casa, perro, casa(forma=cuadrado), carro` → **tres**.

Novedad de la versión 1.1: la unidad 2.2 — la lista

El sistema ya sabe **la lista**, y la sabe de verdad: no de corrido desde el principio, sino contestando **qué va antes y después de una palabra cualquiera**.

```
siguiente: siete      ->  ocho      (por el siete, no por el uno)
anterior: nueve       ->  ocho      (también hacia atrás)
recitar: ocho         ->  ocho, nueve, diez
                          Llego hasta el final de lo que se: no hay más después de 'diez'.
```

Y aquí está la gracia: al llegar al final, no se inventa nada

```
siguiente: diez       ->  no_lo_se
                          'diez' es la ULTIMA palabra que conozco. No me invento la siguiente.
siguiente: once       ->  palabra_desconocida
```

Los niños hacen exactamente esto: en los estudios, entre un 12% y un 25% dicen **«no lo sé»** cuando no saben. Ese «no lo sé» **no es un fallo**: es la respuesta correcta, y es la que este sistema da.

Y sabe revisar una recitación ajena

```
revisar_secuencia: uno,dos,cuatro   ->  orden_saltado   ("falta por decir: tres")
revisar_secuencia: uno,tres,dos     ->  orden_saltado   (los cambió de orden)
revisar_secuencia: uno,dos,once     ->  palabra_desconocida  (se inventó una al final)
revisar_secuencia: uno,dos,tres     ->  correcta
```

Una separación que quiero que sepas (porque es fácil de confundir)

Aquí NO se aprende que «cuatro es uno más que tres». Se aprende que después de «tres» va «cuatro» **porque está en la lista**. La investigación es clara: entender que cada palabra es «una más» (el sucesor) llega **después** de entender la cardinalidad. Así que aquí no se cuela, a propósito.

Novedad de la versión 1.0: la unidad 2.1 — CONTAR

Es la **primera unidad en la que el sistema deduce algo que no se le ha dado**. Hasta ahora, todo lo que contestaba estaba ya escrito en lo que le escribías. Ahora no: le das una colección y él produce el emparejamiento de una palabra con cada objeto.

```
contar: silla_1, silla_2, silla_3

COLECCION:
    silla_1
    silla_2
    silla_3
  EMPAREJAMIENTO (una palabra por objeto, ninguna repetida):
    silla_1    -> uno
    silla_2    -> dos
    silla_3    -> tres
RESULTADO: emparejado
```

Lo segundo, y lo más importante: sabe revisar una cuenta mal hecha

Se le da una colección y una cuenta que ha hecho otro, y el sistema dice si está bien y, si no, cuál es el error. Hay tres errores, y son los tres que la investigación documenta:

```
revisar: a,b,c / uno,uno,tres
  -> se uso la misma palabra mas de una vez: uno
RESULTADO: palabra_repetida          (el error MAS COMUN en los ninos)

revisar: a,b,c / uno,tres,cuatro
  -> falta por decir: dos
RESULTADO: orden_saltado

revisar: a,b,c / uno,dos
  -> quedaron objetos SIN contar: c
RESULTADO: faltan_objetos_sin_contar
```

Esto no se puede hacer por rutina: hay que tener el principio dentro.

Y una honestidad más: no se inventa las palabras que no conoce

El sistema sabe contar hasta **diez**. Si le das más objetos:

```
contar: a,b,c,d,e,f,g,h,i,j,k
RESULTADO: no_alcanza_la_secuencia
  Hay mas objetos que palabras conozco, y NO me invento las que faltan.
  (Aprender mas palabras de la secuencia es la unidad 2.2.)
```

Contar **no es recitar**: es emparejar. Y el sistema solo empareja con lo que tiene.

Lo que todavía NO hace (importante)

Contar no es todavía saber cuántos hay. El sistema **no dice «hay tres»**: pone las palabras, pero todavía no sabe que **la última palabra es el total**. Eso es la **cardinalidad** (unidad 2.4) y llega después — así es como se aprende: primero emparejar, luego entender que la última palabra resume todo.

Novedad de la versión 0.8: la unidad 1.0 — reconocer de un vistazo

Ya se puede enseñar una figura y preguntar cuántos elementos tiene, **de un vistazo**:

```
subitizar: x.x/.../x.x

FIGURA:
    x . x
    . . .
    x . x
  es el patron 'esquinas_cuatro'  ->  CUATRO
RESULTADO: reconocido
```

La figura se escribe con **tres filas separadas por** `/`, donde `x` es un elemento y `.` un hueco.

Lo importante: el sistema no cuenta

Y aquí está la gracia de la unidad. Si la figura **no** es un patrón conocido:

```
subitizar: x../xx./x.x

FIGURA:
    x . .
    x x .
    x . x
RESULTADO: no_reconocido
  No lo reconozco de un vistazo, y NO lo cuento: contar es la unidad 2.3.
  El sistema no adivina: si no lo reconoce, lo dice.
```

Esa figura tiene **cuatro** elementos. El sistema podría contarlos… si supiera contar, y todavía no sabe **(eso es el Nivel 2). Así que no lo finge: dice que no lo reconoce.** Reconocer no es contar.

Y la disposición es lo que hace reconocible a un grupo

- `x.x/.../x.x` → cuatro (esquinas, como un dado) ✔
- `xx./xx./...` → cuatro también: la misma cantidad en otra disposición vale lo mismo ✔
- `x../xx./x.x` → no se reconoce ✘

Esto no es un capricho: la investigación muestra que las disposiciones tipo dado se ven **enteras** y las desordenadas no, y que una fila recta de puntos es difícil de subitizar.

Las cantidades van en **palabras** (`uno`, `cuatro`), no en cifras: las cifras son la unidad 3.1.

Novedad de esta versión: el mismo nombre, dos objetos

Corregido a raíz de una prueba tuya. Antes, esto:

```
carro(color=negro), carro(color=blanco)
```

decía «es **un** objeto» y «se contradicen». Estaba mal, como tú dijiste: una misma cosa no puede ser negra y blanca a la vez**, así que un atributo con** dos valores distintos **es la** prueba de que son **dos objetos**. Ahora:

```
   AVISO: un mismo nombre con datos que NO pueden ser la misma cosa:
     carro  [color=negro]
     carro  [color=blanco]
   -> eso PRUEBA que son objetos DISTINTOS. Quedan asi: carro#1, carro#2
  PLURALIDAD (0.1): son VARIOS OBJETOS
  ATRIBUTOS (0.2): sin_atributo_comun
```

Las tres reglas del nombre, ya juntas:

Escribes	Qué entiende
`casa(color=blanco); casa(forma=cuadrada)`	**La misma casa** (los datos se suman): `casa` blanca y cuadrada
`carro(color=negro); carro(color=blanco)`	**Dos objetos**: `carro#1` y `carro#2` — el color los distingue
`carro, carro`	**Un solo objeto**: sin atributos, nada los distingue

Novedad de esta versión: la unidad 0.3 — pertenencia

Ya se puede preguntar **si un objeto concreto pertenece a un grupo**:

```
pertenece color=rojo: taza(color=rojo); plato(color=azul); vaso(forma=cubo)

   taza       -> PERTENECE
   plato      -> NO_PERTENECE
   vaso       -> NO_DECIDIBLE  (no se sabe su color)
```

Tres respuestas, no dos, y esa es toda la gracia:

- **PERTENECE** — el objeto tiene el color que pide el grupo.
- **NO_PERTENECE** — tiene color, pero **otro**.
- **NO_DECIDIBLE** — **no se sabe**: a ese objeto no le hemos dicho el color. Que no es lo mismo que «no pertenece»: para negar algo hay que saber que es falso, no basta con no saberlo.

Y algo que conviene probar: **un mismo objeto pertenece a varios grupos**. El triángulo verde pertenece a «los verdes» y a «los triángulos» — la pertenencia depende del criterio con el que se pregunta.

El Nivel 0 está completo

Con 0.1, 0.2 y 0.3 cerradas y verificadas, el sistema ya sabe: distinguir objetos, formar grupos por

atributos y decidir pertenencia. **Todo rastreable y todo con casos nuevos probados.**

Novedad de esta versión: cada respuesta dice las DOS cosas

Cuando escribes objetos con atributos, el sistema contesta **en dos capas**:

- **PLURALIDAD (0.1):** cuántas cosas hay — `es UN OBJETO` o `son VARIOS OBJETOS`.
- **ATRIBUTOS (0.2):** si comparten algún atributo, y cuáles.

Así ya no esconde lo que sabe: `casa(color=azul); carro` **no** dice un «indefinido» a secas, sino «son **varios objetos**, y en cuanto a atributos **no se puede decidir** (del color del carro no sabemos nada)».

Y hay una regla nueva, la de la **fusión**:

Escribes	Qué entiende
`casa(color=blanco); casa(forma=cuadrada)`	**La misma casa**, descrita dos veces: sus datos se suman → `casa blanca y cuadrada`, **un solo objeto**
`casa(color=blanco); casa(color=rojo)`	**Se contradicen** (mismo atributo, dos valores) → no elige: `no_decidible`
`casa, casa`	**Un solo objeto** (el nombre manda) — si no, sería un grupo y eso rompería 0.1

Novedad de esta versión: el vocabulario, arreglado

Carlos destapó una contradicción: `casa,carro` decía *grupo*, y `casa(color=blanco); carro(color=negro)` decía *no grupo*. La misma pareja de objetos, respuestas opuestas.

La causa era usar la palabra **grupo** con dos sentidos distintos. Ya está separado:

Unidad	La pregunta que responde	Respuestas
0.1	¿hay **una** cosa o **varias**? (sin contar)	`un_objeto` / `varios_objetos`
0.2	¿esos objetos **comparten** algo?	`grupo_por_atributo` / `sin_atributo_comun` / `una_sola_entidad`

La palabra GRUPO es solo de 0.2 y siempre quiere decir: varios objetos que comparten un atributo con el mismo valor. Así ya no se contradicen:

```
casa,carro                                -> varios_objetos
casa(color=blanco); carro(color=negro)    -> varios objetos, pero sin_atributo_comun
casa_A(color=blanco); casa_B(color=blanco)  -> grupo_por_atributo, comparten color
```

En una línea

Doble clic en `E:\Euclides\PROBAR.bat`. Se abre una ventana, corre tus casos y te deja escribir los tuyos. Para salir, escribe `salir`.

Qué sabe hacer hoy (y qué no)

0.1	Decidir si hay un objeto o varios objetos (sin contarlos)
0.2	**Reconocer** si varios objetos forman grupo y decir **qué atributos comparten**
0.2	**Formar** el grupo de los que cumplen un criterio (`color=rojo`)
0.3	**Pertenencia**: si un objeto concreto pertenece a un grupo (`pertenece` / `no_pertenece` / `no se sabe`)
1.0	**Subitizing**: reconocer de un vistazo la cantidad de un grupo pequeño, sin contar
2.1	**Contar**: emparejar una palabra con cada objeto, en orden (`emparejado`)
2.1	**Revisar una cuenta**: decir si está bien hecha y **cuál** de los tres errores tiene
2.2	**La lista**: qué va antes y después de cualquier palabra, sin empezar por «uno»
2.2	**Al llegar al final dice** `no_lo_se`: no se inventa palabras nuevas
2.3	**Cuántos hay**: cuenta la colección y da la cantidad (la última palabra del recuento)
2.4	**Dame N**: produce un conjunto con esa cantidad (`dame: tres`) y **lo verifica contándolo**
2.4	**¿Tiene N?**: `tiene: a,b,c / tres` → sí o no (y dice cuál tiene)
2.5	**El orden no importa**: `orden: a,b,c` → coinciden (y enseña los dos recuentos)
2.5	**Juzga un conteo ajeno**: `conteo:` → válido, o cuál de las reglas lógicas está rota
2.6	**Cualquier cosa se cuenta**: `misma_cantidad: silla_1,silla_2 / sonido_1,sonido_2` → sí
3.1	**Las cifras**: `cifra: tres` → 3; `palabra: 3` → `tres`; `cifra_de: a,b,c` → 3
3.2	**El cero**: `cuantos:` con la colección vacía → `cero`; `cifra_de:` → `0`
3.3	**Las dos listas**: `listas` → las enseña. `anterior: uno` → `cero`; `siguiente: cero` → `uno`
3.4	**El puente**: `caminos` → los seis caminos; `puente: a,b,c` → 3; `dame: a,b,c,d,e / 3` → lo coge
4.1	**Comparar cantidades**: `comparar: a,b,c / d,e` → `mas`; el tamaño de los objetos **no** cuenta
4.2	**Comparar números**: `comparar_cifras: 3 / 2` → >; `2 / 3` → <; `4 / 4` → =

0.1	Decidir si hay un objeto o varios objetos (sin contarlos)
5.1	**Partes y todo**: `descomponer: a,b,c,d,e` → `0+5,1+4,2+3`; `componer: a,b / c,d,e` → `cinco`
5.2	**La suma**: `sumar: a,b,c / d,e` → `3 + 2 = 5`; `anadir: a,b,c / d,e` → `3 + 2 = 5` (otra historia)
Siempre	Contesta con **las dos capas**: cuántas cosas hay y si comparten atributos
Siempre	Si no entiende lo que le escribes, **lo dice**; nunca revienta
Siempre	Si no tiene bastante (más objetos que palabras, o un dato que no sabe), **lo dice**; no se lo inventa
No sabe todavía	**Decir cuántos hay** («hay tres»). El sistema empareja, pero aún no sabe que la última palabra es el total (eso es la 2.4). Tampoco compara cantidades ni usa cifras

Las tres formas de preguntar

1. Un objeto o varios (unidad 0.1)

Cosas separadas por **comas**. Si repites un nombre, es **la misma** cosa señalada otra vez.

```
silla                -> un_objeto
silla,mesa           -> varios_objetos
silla,silla          -> un_objeto   (la misma silla, dos veces)
```

2. ¿Forman grupo? (unidad 0.2)

Objetos con atributos. Valen **punto y coma** o **comas de fuera** de los paréntesis. Los atributos van entre paréntesis como `nombre=valor`, separados por comas.

```
caja_A(color=rojo,forma=cubo); caja_B(color=rojo,forma=esfera)   -> grupo por color
caja_A(color=rojo,forma=cubo), caja_B(color=azul,forma=cubo)     -> grupo por forma
casa(color=blanco); carro(color=negro)                           -> sin_atributo_comun
```

3. Dame el grupo de... (unidad 0.2)

El criterio, dos puntos, y la escena.

```
color=rojo: a(color=rojo,forma=cubo); b(color=azul,forma=cubo); c(color=rojo,forma=esfera)
   -> el grupo son a y c
```

4. ¿Pertenece? (unidad 0.3)

Empieza por `pertenece`, luego el criterio del grupo, dos puntos, y el objeto o los objetos.

```
pertenece color=rojo: taza(color=rojo)
   -> taza -> PERTENECE

pertenece color=rojo: taza(color=rojo); plato(color=azul); vaso(forma=cubo)
   -> taza  -> PERTENECE
   -> plato -> NO_PERTENECE
   -> vaso  -> NO_DECIDIBLE  (no se sabe su color)
```

5. ¿Cuántos hay, de un vistazo? (unidad 1.0)

Empieza por `subitizar`, dos puntos, y la figura: **tres filas separadas por** `/`, con `x` de elemento y `.` de hueco.

```
subitizar: x.x/.../x.x
   -> es el patron 'esquinas_cuatro' -> CUATRO

subitizar: x../xx./x.x
   -> no_reconocido  (no lo reconoce de un vistazo, y NO lo cuenta)
```

6. Contar (unidad 2.1)

Empieza por `contar`, dos puntos, y la colección (separada por comas).

```
contar: silla_1, silla_2, silla_3
   -> silla_1 -> uno ; silla_2 -> dos ; silla_3 -> tres   (emparejado)

contar: silla_1, silla_1, silla_1
   -> un solo objeto (es el mismo nombre tres veces) -> una sola palabra: uno

contar: casa(color=blanco), casa, perro, casa(forma=cuadrado), carro
   -> casa [color=blanco, forma=cuadrado]  -> uno     (las tres menciones son LA MISMA casa: los
   -> perro                                -> dos      datos se suman, y no dan tres palabras)
   -> carro                                -> tres
```

7. Revisar una cuenta (unidad 2.1)

Empieza por `revisar`, la colección, una **barra** `/`, y las palabras que le pusieron.

```
revisar: a,b,c / uno,dos,tres      -> correcta
revisar: a,b,c / uno,uno,tres      -> palabra_repetida
revisar: a,b,c / uno,tres,cuatro   -> orden_saltado
revisar: a,b,c / uno,dos           -> faltan_objetos_sin_contar
revisar: a,b / uno,once            -> palabra_desconocida
revisar: a,b / uno,dos,tres        -> palabras_de_mas
```

8. La lista: qué va antes y después (unidad 2.2)

```
siguiente: tres     -> cuatro
anterior: tres      -> dos
siguiente: diez     -> no_lo_se    (la lista se acaba; no se inventa «once»)
siguiente: once     -> palabra_desconocida
recitar: ocho       -> ocho, nueve, diez
recitar: uno        -> la lista entera, uno a uno
```

9. Revisar una recitación ajena (unidad 2.2)

```
revisar_secuencia: uno,dos,tres     -> correcta
revisar_secuencia: uno,dos,cuatro   -> orden_saltado
revisar_secuencia: uno,tres,dos     -> orden_saltado
revisar_secuencia: uno,dos,once     -> palabra_desconocida
```

10. Cuántos hay (unidad 2.3)

```
cuantos: cosa                       -> uno
cuantos: silla_1, silla_2, silla_3  -> tres
cuantos: a,b,c,d,e                  -> cinco
cuantos: a,a,a,a,a                  -> uno   (el mismo objeto cinco veces)
```

```
cuantos: (once objetos)              -> no_alcanza_la_secuencia
```

11. Dame N (unidad 2.4) — le das el montón

```
dame: a,b,c,d,e / tres   ->  coge a, b, c ; deja d, e   (y lo verifica contándolo)
dame: a,b,c / uno        ->  coge a ; deja b, c
dame: a,b / tres         ->  no_hay_bastantes  (no se inventa el tercero)
dame: tres               ->  NO ENTIENDO: dame necesita un montón del que coger
```

12. ¿Tiene N? (unidad 2.4)

```
tiene: a,b,c / tres     -> si
tiene: a,b,c / cinco    -> no   (tiene tres)
tiene: (once objetos) / cinco -> no_alcanza_la_secuencia
```

13. El orden no importa (unidad 2.5)

```
orden: a,b,c     -> coinciden (y enseña el recuento al derecho y al revés)
orden: cosa      -> coinciden (un solo objeto)
```

14. Juzgar un conteo ajeno (unidad 2.5)

```
conteo: a,b,c / uno,dos,tres          -> valido
conteo: c,a,b / uno,dos,tres          -> valido   (empezó por el tercero: está permitido)
conteo: e,d,c,b,a / uno,dos,tres,cuatro,cinco -> valido  (todo al revés)
conteo: a,b,b,c / uno,dos,tres,cuatro -> objeto_repetido  (contó b dos veces: NO vale)
conteo: a,b,c / uno,dos,dos           -> palabra_repetida
conteo: a,b,c / uno,tres,cuatro       -> orden_saltado
```

15. ¿La misma cantidad? (unidad 2.6)

```
misma_cantidad: silla_1,silla_2,silla_3 / sonido_1,sonido_2,sonido_3   -> si
misma_cantidad: casa(color=rojo),perro / idea,salto                    -> si
misma_cantidad: a_1,a_2 / b_1,b_2,b_3                                  -> no
misma_cantidad: silla,silla,silla / b_1,b_2,b_3                        -> no
        (OJO: «silla,silla,silla» es UNA silla, no tres)
```

16. Las cifras (unidad 3.1)

```
cifra: tres      -> 3        (de la palabra a la cifra)
palabra: 3       -> tres     (de la cifra a la palabra)
cifra_de: a,b,c  -> 3        (de la cantidad a la cifra)

cifra: diez      -> sin_cifra         («diez» es palabra conocida, pero su cifra son DOS símbolos)
palabra: 0       -> cifra_desconocida (el cero todavía no está)
palabra: 10      -> cifra_desconocida (dos símbolos)
cifra_de: (diez objetos) -> no_alcanza_las_cifras
```

17. El cero (unidad 3.2)

```
cuantos:         -> cero   (la colección está vacía: no hay ninguno)
cifra_de:        -> 0
cifra: cero      -> 0
palabra: 0       -> cero

cuantos: cosa    -> uno    (con un objeto NO es cero)
```

OJO: el 0 de `palabra: 10` → `cifra_desconocida`, porque ese cero es **posicional** (Sistema Decimal).

18. Las dos listas (unidad 3.3)

```
listas                -> las dos listas y en qué se diferencian

anterior: uno         -> cero      (en los NUMEROS; en la lista de CONTAR el uno es el primero)
siguiente: cero       -> uno
anterior: cero        -> no_lo_se  (el cero es el primero de los números)
siguiente: nueve      -> no_lo_se  (el nueve es el último de los números)
siguiente: diez       -> sin_numero ("diez" está en la de contar, no en la de números)
recitar: cero         -> palabra_desconocida (recitar es la lista de CONTAR: empieza en uno)
```

19. El puente (unidad 3.4)

```
caminos                     -> los seis caminos entre cantidad, palabra y cifra

puente: a,b,c               -> 3      (te enseña el recuento y las tres representaciones)
puente:                     -> 0      (colección vacía)
puente: a,...,j (diez)      -> diez   (hay palabra, pero NO cifra: el 10 son dos símbolos)

dame: a,b,c,d,e / 3         -> lo coge  (el camino que faltaba: de la CIFRA a los objetos)

subitizar: x.x/.../x.x      -> CUATRO, y su CIFRA es 4 (el camino CORTO: no cuenta)
subitizar: x../xx./x.x      -> no_reconocido (y NO cuenta a escondidas)
```

20. Comparar cantidades (unidad 4.1)

```
comparar: a,b,c / d,e       -> mas          (la PRIMERA tiene más)
comparar: a,b / c,d,e       -> menos
comparar: a,b / c,d         -> tantos_como  (las dos lo mismo)
comparar: /                 -> tantos_como  (vacía y vacía: cero y cero)
comparar: a,b,c             -> NO ENTIENDO  (sin la barra no son dos)

comparar: g1(tamano=grande), g2(tamano=grande) / p1(tamano=pequeno), p2(tamano=pequeno), p3(tamano=pequeno)
                            -> menos        (¡el tamaño NO cuenta!)
```

21. Comparar números (unidad 4.2)

```
comparar_cifras: 3 / 2      -> >            (tres es mayor que dos)
comparar_cifras: 2 / 3      -> <
comparar_cifras: 4 / 4      -> =
comparar_cifras: 0 / 1      -> <            (el cero también se compara)
comparar_cifras: 9 / 7      -> >
comparar_cifras: 10 / 2     -> cifra_desconocida  (el 10 son DOS símbolos)
comparar_cifras: 3 / tres   -> cifra_desconocida  ('tres' no es una cifra)
comparar_cifras: 3          -> NO ENTIENDO        (comparar son DOS)
```

22. Componer y descomponer (unidad 5.1)

```
descomponer: a,b,c,d,e          -> 0+5,1+4,2+3    (cinco, TRES maneras)
descomponer: a,b,c,d            -> 0+4,1+3,2+2
descomponer: a                  -> 0+1
descomponer:                    -> el_cero_no_se_parte
descomponer: (diez objetos)     -> 0+diez,1+9,2+8,3+7,4+6,5+5   (seis maneras)

componer: a,b / c,d,e           -> cinco          (dos y tres son cinco)
componer: a,b,c / d,e           -> cinco          (¡el orden no importa!)
componer: / a,b,c,d             -> cuatro         (el cero también es una parte)
componer: a,b,c,d,e,f / g,h,i,j -> diez           (justo el límite)
componer: a,b,c,d,e,f / g,h,i,j,k -> no_alcanza   (seis y cinco son once)
componer: a,b,c                 -> NO ENTIENDO    (hacen falta DOS partes)
```

23. La suma (unidad 5.2)

```
sumar: a,b,c / d,e              -> 3 + 2 = 5      (AGREGACION: juntar dos montones que ya estan)
anadir: a,b,c / d,e             -> 3 + 2 = 5      (AUMENTACION: el monton crece)
sumar: a,b / c,d,e              -> 2 + 3 = 5      (respeta el orden que le das)
sumar: / a,b,c,d                -> 0 + 4 = 4      (con una parte vacía)
sumar: a,b,c,d,e,f / g,h,i,j    -> 6 + 4 = diez   (el límite; y la cifra del 10 no la tiene)
sumar: a,b,c,d,e,f / g,h,i,j,k  -> no_alcanza     (pasa de diez)
sumar: a,b,c                    -> NO ENTIENDO    (hacen falta DOS cantidades)
```

Lo más importante: el nombre es el identificador

El sistema **no ve** los objetos, y **no adivina** por el parecido de los nombres. El nombre que escribes **es** la identidad de la cosa.

- `silla` y `silla` → **la misma** silla.
- `silla_A` y `silla_B` → **dos** sillas distintas, aunque en el mundo las dos se llamen igual.

Y si escribes la misma cosa con dos descripciones que no pueden ser ciertas a la vez:

```
casa(color=azul); casa(color=rojo)
   -> son DOS objetos distintos: el color los distingue (casa#1 y casa#2)
```

Pero si los datos **no** se contradicen, son la misma cosa y sus datos se **suman**:

```
casa(color=blanco); casa(forma=cuadrada)
   -> un solo objeto: casa blanca y cuadrada
```

Si escribes algo que no entiende

No revienta: te lo dice y te recuerda las formas correctas.

```
pregunta> caja(colorrojo)

NO ENTIENDO ESTO: no entiendo el atributo 'colorrojo'
   se escribe asi:  nombre=valor

FORMAS QUE SI ENTIENDO:
  0.1  un objeto o varios  silla,mesa
  0.2  forman grupo?       a(color=rojo,forma=cubo); b(color=azul,forma=cubo)
  0.2  dame el grupo de... color=rojo: a(color=rojo); b(color=azul)

El sistema no adivina: si no entiende, lo dice.
```

Las tres formas de lanzarlo

1. **Doble clic en** `PROBAR.bat` — lo normal. Corre tus casos y abre la ventana.
2. `py -3 E:\Euclides\probar.py "silla,mesa"` — una pregunta suelta.
3. `py -3 E:\Euclides\probar.py --casos` — todos tus casos de golpe.

Órdenes dentro de la ventana: `casos`, `ayuda`, `salir`.

Dónde escribes TUS casos

Archivo: `E:\Euclides\mis_casos.txt` (Bloc de notas). Cada línea:

```
pregunta | lo que espero | comentario
```

Lo que puedes poner en "lo que espero":

Escribes	Significa
`objeto` o `uno`	una sola entidad
`varios`	dos o más entidades distintas
`grupo:color`	forman grupo y comparten el color
`grupo:color,forma`	comparten las dos cosas
`sin_atributo_comun`	son varios, pero no comparten nada
`una_sola_entidad`	solo hay una cosa (o es la misma descrita dos veces): la pregunta de grupo no aplica
`tiene:a,c`	al formar un grupo, el resultado son esos
`no_hay_grupo`	ninguno cumple el criterio
`no_decidible`	el sistema debe decir que no sabe
`no_entendido`	el sistema debe decir que no entiende la escritura
`taza:pertenece`	(0.3) la taza pertenece al grupo que se pregunta
`plato:no_pertenece`	(0.3) el plato no pertenece
`vaso:no_decidible`	(0.3) del vaso no se sabe: no es lo mismo que no pertenecer
`reconocido:cuatro`	(1.0) la figura es el patrón de cuatro
`no_reconocido`	(1.0) el sistema no reconoce la figura de un vistazo, y no la cuenta
`emparejado`	(2.1) contó: una palabra por objeto, sin repetir ni saltar
`no_alcanza_la_secuencia`	(2.1) hay más objetos que palabras conoce: no se las inventa
`correcta`	(2.1) la cuenta que le dieron está bien hecha
`palabra_repetida`	(2.1) la misma palabra para dos objetos
`orden_saltado`	(2.1) se saltaron una palabra de la secuencia
`faltan_objetos_sin_contar`	(2.1) algún objeto se quedó sin contar
`palabra_desconocida`	(2.1) hay una palabra que no está en la secuencia
`palabras_de_mas`	(2.1) hay más palabras que objetos
`cuatro` / `ocho` / …	(2.2) la palabra que va después (o antes)
`no_lo_se`	(2.2) la lista se acaba ahí: el sistema no se inventa la siguiente
`recitado:ocho,nueve,diez`	(2.2) la recitación desde esa palabra
`uno` / `tres` / `cinco` …	(2.3) la cantidad que hay. **OJO**: aquí `uno` es la cantidad, no «un objeto»
`no_alcanza_la_secuencia`	(2.3) más objetos que palabras conoce: no se inventa la cantidad

Escribes	Significa
`cumplido`	(2.4) `dame: <montón> / N` ha cogido N del montón y lo ha verificado
`no_hay_bastantes`	(2.4) en el montón hay menos de los pedidos: no se inventa los que faltan
`si` / `no`	(2.4) `tiene: <colección> / N` — si el conjunto tiene esa cantidad o no
`coinciden`	(2.5) contar en otro orden da la misma cantidad
`valido`	(2.5) el recorrido de conteo está bien (aunque empiece por donde quiera)
`objeto_repetido`	(2.5) contó dos veces el mismo objeto: la «compensación», NO vale
`si` / `no`	(2.6) las dos colecciones tienen la misma cantidad o no
`1` … `9`	(3.1) la cifra correspondiente
`sin_cifra`	(3.1) la palabra existe pero su cifra son dos símbolos (p. ej. `diez`)
`cifra_desconocida`	(3.1) esa cifra no está entre las del 1 al 9
`no_alcanza_las_cifras`	(3.1) la cantidad no tiene cifra entre las conocidas
`cero`	(3.2) la colección está vacía: la cantidad es ninguna
`0`	(3.2) la cifra del cero
`sin_numero`	(3.3) la palabra existe, pero no tiene sitio en la secuencia de números (p. ej. `diez`)
`puente_sin_cifra`	(3.4) la cantidad no tiene cifra: el caso del `diez` (da la palabra)
`cero_no_se_coge`	(3.4) `dame: ... / 0`: dar es coger, y el vacío no se coge
`los_seis_caminos`	(3.4) al escribir `caminos`
`mas`	(4.1) la **primera** colección tiene más
`menos`	(4.1) la primera tiene menos
`tantos_como`	(4.1) las dos tienen lo mismo
`no_alcanza`	(4.1) alguna colección pasa de diez objetos
`>`	(4.2) la primera cifra es mayor
`<`	(4.2) la primera cifra es menor
`=`	(4.2) las dos cifras son iguales
`cifra_desconocida`	(4.2) alguna no es una cifra del 0 al 9
`N maneras`	(5.1) cuántas maneras de partir en dos (p. ej. `3 maneras`)
`el_cero_no_se_parte`	(5.1) `descomponer:` con la colección vacía
`3 + 2 = 5`	(5.2) la expresión de la suma
`3 + 7 = diez`	(5.2) cuando el total es diez: da la palabra
`?`	no comprobar nada, solo ver qué responde

Guarda el archivo y vuelve a lanzar el probador: **se recarga solo**. Los # son comentarios.

Por qué el sistema explica lo que dice

Cada respuesta sale con su **traza**: qué reglas se han probado, en qué orden, cuál ha decidido y qué conocimiento escrito la respalda. Ejemplo real:

```
PREGUNTA (0.2): forman GRUPO estos objetos?
ESCENA:
    casa  [color=blanco]
    carro  [color=negro]
PLURALIDAD (0.1): son VARIOS OBJETOS
  reglas probadas, en orden:
    - reconocer_escena_vacia : escena_vacia(escena) -> falso
    - reconocer_una_sola_entidad : menos_de_dos_entidades(escena) -> falso
    - reconocer_atributo_indeterminado : algun_atributo_indeterminado(escena) -> falso
    - reconocer_grupo_por_atributo : hay_atributo_compartido_por_todos(escena) -> falso
    - reconocer_sin_atributo_comun : sin_atributo_compartido_por_todos(escena) -> cierto
ATRIBUTOS (0.2): sin_atributo_comun
  lo decide la regla 'reconocer_sin_atributo_comun'
  respaldado por el conocimiento 'grupo_por_atributo' (Grupo por atributo) [FIJADO]
```

Qué buscar cuando lo pruebes

1. **Que 0.1 y 0.2 ya no se contradigan**: `casa,carro` contra `casa(color=blanco); carro(color=negro)`.
2. **La pertenencia y sus tres respuestas**: `pertenece color=rojo: taza(color=rojo); plato(color=azul); vaso(forma=cubo)`.

Comprueba que el vaso dice **«no se sabe»** y no «no pertenece».

3. **El mismo objeto en dos grupos**: `pertenece color=verde:` y `pertenece forma=triangulo:` sobre `triangulo(color=verde,forma=triangulo)`. Debería pertenecer a los dos.
4. **La fusión y la distinción**: `casa(color=blanco); casa(forma=cuadrada)` debería decir que es la misma casa **con los datos sumados; y** `carro(color=negro); carro(color=blanco)` **debería dar** dos objetos (`carro#1`, `carro#2`), no «no se sabe».
5. **Cosas escritas a medias o mal** (`caja(colorrojo)`, `caja(color=)`, `pertenece =rojo: ...`): todas deberían contestar "no entiendo", ninguna debería romperse.
6. **La misma cosa con dos descripciones distintas que sí chocan** (`casa(color=azul); casa(color=rojo)`).
7. **Un objeto al que le falta un atributo** que los demás sí tienen: debería callarse **en atributos** pero seguir diciendo que son varios objetos.
8. **Repetir el mismo nombre** en 0.1 (`taza,taza,taza`).
9. **Pedir un grupo que no existe** (`color=verde:` cuando nadie es verde).
10. **El subitizing y su negativa**: `subitizar: x.x/.../x.x` debería dar **cuatro**; y `subitizar: x../xx./x.x` (que también tiene cuatro) debería dar **no_reconocido** — porque el sistema **no cuenta**. Ese "no" es la respuesta correcta, no un fallo.
11. **(2.1) Que cuente una colección**: `contar: a,b,c` debería emparejar uno, dos y tres.

12. **(2.1) La prueba que encontraste tú**: `contar: casa(color=blanco), casa, perro, casa(forma=cuadrado), carro` debería dar **tres** palabras (casa, perro, carro), y la casa con sus dos datos sumados. Y el contraste: `contar: casa(color=blanco), casa(color=rojo), carro` sí son **dos** casas, porque el color las distingue.
13. **(2.1) Que cace los tres errores**, uno por uno: `revisar: a,b,c / uno,uno,tres` (repetida — el más común), `revisar: a,b,c / uno,tres,cuatro` (salto) y `revisar: a,b,c / uno,dos` (un objeto sin contar). Y que diga **cuál** es.
14. **(2.1) Que NO se invente palabras**: `contar:` con once objetos debería decir `no_alcanza_la_secuencia`, no seguir contando.
15. **(2.1) Que la cuenta correcta se reconozca como correcta**: `revisar: a,b,c / uno,dos,tres`.
16. **(2.2) Que sepa la lista sin empezar por uno**: `siguiente: siete` → ocho, y `anterior: nueve` → ocho.
17. **(2.2) Que al llegar al final NO se invente nada**: `siguiente: diez` → `no_lo_se`, y `siguiente: once` → `palabra_desconocida`. Ese «no lo sé» es la respuesta correcta.
18. **(2.2) Que cace una recitación mal dicha**: `revisar_secuencia: uno,dos,cuatro` → `orden_saltado` y que diga que falta el tres; `revisar_secuencia: uno,dos,once` → `palabra_desconocida`.
19. **(2.3) Que diga cuántos hay**: `cuantos: silla_1, silla_2, silla_3` → `tres`; `cuantos: a,b,c,d,e` → `cinco`; `cuantos: cosa` → uno.
20. **(2.3) Que no cuente dos veces el mismo objeto**: `cuantos: a,a,a,a,a` → uno.
21. **(2.3) Que no se invente la cantidad**: `cuantos:` con once objetos → `no_alcanza_la_secuencia`.
22. **(2.4) Que sepa dar, desde el montón que le des**: `dame: a,b,c,d,e / tres` → coge tres y deja dos, y lo verifica.
23. **(2.4) Que no se invente objetos**: `dame: a,b / tres` → `no_hay_bastantes`; y `dame: tres` (sin montón) → «NO ENTIENDO: dame necesita un MONTON».
24. **(2.4) Que confirme y rechace**: `tiene: a,b,c / tres` → `si`; `tiene: a,b,c / cinco` → no.
25. **(2.5) Que el orden no importe**: `orden: a,b,c` → `coinciden` (y debería enseñarte los dos recuentos).
26. **(2.5) Que acepte un conteo empezado por donde sea**: `conteo: c,a,b / uno,dos,tres` → `valido`.
27. **(2.5) Y que RECHAce la compensación**: `conteo: a,b,b,c / uno,dos,tres,cuatro` → `objeto_repetido`. Este es el bueno: el conteo **acaba en «cuatro»**, que es el número correcto de objetos, y aun así **no vale**, porque contó uno dos veces y otro ninguna.
28. **(2.6) Que cuente cualquier cosa**: `misma_cantidad: silla_1,silla_2 / sonido_1,sonido_2` → `si`.
29. **(2.6) Y la trampa de los nombres repetidos**: `misma_cantidad: silla,silla,silla / b_1,b_2,b_3`

→ no, porque `silla,silla,silla` es **una** silla. (Para tener tres hay que escribir `silla_1,silla_2,silla_3`.)

30. **(3.1) Que sepa las cifras**: `cifra: uno` → `1`, `cifra: nueve` → `9`, `palabra: 3` → `tres`, `cifra_de: a,b,c` → `3`.
31. **(3.1) Que sepa dónde acaba**: `cifra: diez` → `sin_cifra`; `palabra: 10` → `cifra_desconocida`; `cifra_de:` con diez objetos → `no_alcanza_las_cifras`. (`palabra: 0` ya **sí** es una cifra: es el cero, unidad 3.2.)
32. **(3.2) Que el vacío sea cero**: `cuantos:` → `cero`; `cifra_de:` → `0`; `cifra: cero` → `0`; `palabra: 0` → `cero`.
33. **(3.2) Y que cero no sea «no se sabe»**: `cuantos: cosa` → uno (no cero); y `tiene: (vacío) / uno → no_decidible` (ahí el cero **no** entra, a propósito).
34. **(3.3) Las dos listas**: escribe `listas`. La de contar debe empezar en **uno** y llegar a **diez**; la de números debe empezar en **cero** y acabar en **nueve**.
35. **(3.3) El arreglo**: `anterior: uno` → `cero`; `siguiente: cero` → uno; `anterior: cero` → `no_lo_se`; `siguiente: nueve` → `no_lo_se`.
36. **(3.3) Que distinga «no existe» de «no tiene sitio»**: `siguiente: diez` → `sin_numero`, pero `siguiente: once` → `palabra_desconocida`.
37. **(3.3) Que recitar no se confunda**: `recitar: cero` → `palabra_desconocida` (y te explica por qué).
38. **(3.4) Los seis caminos**: escribe `caminos`. Y prueba cada uno: `cuantos: a,b,c` → `tres`; `dame: a,b,c,d / tres` → lo coge; `cifra_de: a,b,c` → 3; `dame: a,b,c,d / 3` → lo coge; `cifra: tres` → 3; `palabra: 3` → `tres`.
39. **(3.4) El puente**: `puente: a,b,c` → 3. `puente: silla_1,silla_2` → 2. `puente:` → `0`.
40. **(3.4) Los dos caminos**: `subitizar: x.x/.../x.x` → CUATRO **y su cifra 4**, diciendo que no ha contado. Y `subitizar: x../xx./x.x` → `no_reconocido` (no lo inventa).
41. **(3.4) Que diga lo que NO tiene**: `puente:` con diez objetos (`a,b,c,d,e,f,g,h,i,j`) → `diez`, pero avisando de que **la cifra no la tiene** (el 10 son dos símbolos).
42. **(3.4) Y que no coja el vacío**: `dame: a,b,c,d / 0` → `cero_no_se_coge`.
43. **(4.1) Que compare**: `comparar: a,b,c / d,e` → `mas`; `comparar: a,b / c,d,e` → `menos`; `comparar: a,b / c,d` → `tantos_como`.
44. **(4.1) La trampa del tamaño**: `comparar: g1(tamano=grande), g2(tamano=grande) / p1(tamano=pequeno), p2(tamano=pequeno), p3(tamano=pequeno) → menos`. **Dos grandes NO son más que tres pequeñas** — y el sistema te avisa de que el tamaño no cuenta.
45. **(4.1) El cero y la identidad**: `comparar: /` → `tantos_como`; `comparar: / a` → `menos`; `comparar: casa(color=rojo), casa(forma=cuadrado), carro / x,y` → `tantos_como` (la casa repetida es **una**).
46. **(4.1) Que no confunda**: `comparar: a,b,c` (sin barra) → `NO ENTIENDO` (comparar son dos).

47. **(4.2) Los símbolos**: `comparar_cifras: 3 / 2` → >; `comparar_cifras: 2 / 3` → <; `comparar_cifras: 4 / 4` → =.

48. **(4.2) Que cuadre con lo que ya sabía**: `comparar_cifras: 3 / 2` → > **y** en la explicación tiene que decir que «3 cosas frente a 2 cosas» y que los dos caminos dicen lo mismo. Ahí está lo importante: el símbolo no es una regla suelta.

49. **(4.2) Trampas**: `comparar_cifras: 10 / 2` → `cifra_desconocida` (el 10 son dos símbolos); `comparar_cifras: 3 / tres` → `cifra_desconocida`; `comparar_cifras: 3` → `NO ENTIENDO`.

50. **(5.1) Partir**: `descomponer: a,b,c,d,e` → `0+5,1+4,2+3`. Y **lee la explicación**: tiene que decirte que NO repite la pareja al revés y que el cero es una parte válida.

51. **(5.1) Que cuente bien**: `descomponer: a,b,c,d` → `0+4,1+3,2+2`; `descomponer: a` → `0+1`.

52. **(5.1) Juntar**: `componer: a,b / c,d,e` → `cinco`. Y `componer: a,b,c / d,e` → `cinco` también (**el orden no importa**).

53. **(5.1) Trampas**: `descomponer:` → `el_cero_no_se_parte`; `componer: a,b,c,d,e,f / g,h,i,j,k` → `no_alcanza`; `componer: a,b,c` → `NO ENTIENDO`.

54. **(5.2) Sumar**: `sumar: a,b,c / d,e` → `3 + 2 = 5`. Y **lee la explicación**: tiene que decirte que el = significa «son lo mismo» y **no** «pon el resultado».

55. **(5.2) Las dos estructuras**: `sumar: a,b,c / d,e` sale como **AGREGACIÓN**, y `anadir: a,b,c / d,e` como **AUMENTACIÓN** — con el **mismo** `3 + 2 = 5`.

56. **(5.2) Que respete tu orden**: `sumar: a,b / c,d,e` → `2 + 3 = 5` (no reordena por su cuenta).

57. **(5.2) Trampas**: `sumar: a,b,c,d,e,f / g,h,i,j,k` → `no_alcanza`; `sumar: a,b,c` → `NO ENTIENDO`.

58. Cosas tuyas: nombres con espacios, mayúsculas y minúsculas, números (`caja1`, `caja2`).

Con esta unidad queda cerrado el **Nivel 2 (conteo)**: correspondencia, secuencia, cantidad, cardinalidad, orden y abstracción — **las seis, con pruebas en verde y con prueba de dominio**.

Lo que el sistema sabe hacer hoy, de principio a fin:

- distinguir objetos y su identidad, agrupar por atributos, decidir pertenencia (Nivel 0);
- contar una colección emparejando una palabra por objeto (2.1);
- saber la lista de contar y **saber dónde acaba** (2.2);
- decir cuántos hay (2.3);
- **producir** la cantidad que le pidas, cogiéndola de un montón (2.4);
- juzgar si un conteo ajeno está bien hecho, distinguiendo **reglas lógicas de convencionales** (2.5);
- contar **cualquier cosa**, sea del tipo que sea (2.6).

Y todo con la misma disciplina: cada respuesta sale con su traza, cada regla está escrita, y cuando no sabe algo **lo dice** en vez de inventárselo.

Si algo te chirría, dímelo con **la línea exacta que escribiste y lo que esperabas**.

Los archivos, por si quieres mirar

Archivo	Qué es
`E:\Euclides\conocimiento\*.kn`	El saber, en texto plano. Se puede leer y editar
`E:\Euclides\sistema\motor.py`	El programa que lee, decide y explica
`E:\Euclides\pruebas\`	Las pruebas del proyecto (0.1, 0.2, 0.3, 1.0, 2.1 y robustez)
`E:\Euclides\mis_casos.txt`	**Tus casos**
`E:\Euclides\PROBAR.bat`	El lanzador de doble clic
`E:\Euclides\PROGRESO.md`	Qué está hecho, qué falla, qué toca
`E:\Euclides\diario\`	Lo que se hizo cada día y lo que se midió